SOLID 检查清单（Phase 3 任务 1）

背景

按照作业要求，先由 AI 基于 P1 需求规格说明书和 P2 架构设计文档生成完整类图，再由团队对照 SOLID 原则逐条审查，记录 AI 原始设计中的违规点并制定修正方案。

> 说明：「AI 原始设计」指 AI 第一版生成的类图；「修正后」指团队审查并修改后的最终版本（即提交的类图）。

一、单一职责原则（Single Responsibility Principle, SRP）

检查问题

是否存在类承担过多职责的情况？

检查项	内容
AI 是否违反	是（2 处）
违反说明	1. `OrderService`：同时负责接单、确认完成、取消订单、计算信用分变动、发送站内通知（5 项职责）。信用分属于「用户信用」子域、通知属于「消息」子域，不应与订单状态流转耦合。 2. `DemandService`：既负责需求 CRUD，又负责标签（Tag）增删改查。标签是

	独立子域且被 Demand、User 等多实体共用，不应内嵌在 DemandService 中。
修正方案	1. 拆分职责：将信用分逻辑抽至 `CreditService`（含 addCredit/deductCredit/canAcceptOrder 方法），通知发送抽至 `NotificationService`，`OrderService` 仅保留订单状态流转和业务编排； 2. 拆分标签管理：抽至 `TagService`，`DemandService` 仅在创建需求时通过 `DemandFactory` 关联标签。

二、开闭原则（Open/Closed Principle, OCP）

检查问题

新增需求类型（板块）或订单状态时，是否需要修改现有代码？

检查项	内容
AI 是否违反	是（2 处）
违反说明	1. 需求创建：`DemandService.createDemand()` 内部通过 `switch(category)` 判断需求类型（ERRAND/ SKILL/ STUDY/ SECONDHAND）并做差异化校验，新增类型（如「表白墙」）需修改该 switch 块；

	2. 订单状态流转：`OrderService` 的接单、确认、取消等方法通过 `if-else` 链判断 `OrderStatus`，新增状态（如「争议中」）需在多个方法中新增分支。
修正方案	1. 需求创建：引入**工厂模式** - 定义 `DemandValidator` 接口，为 ERRAND/SKILL/STUDY/SECONDHAND 各实现一个校验类； - `DemandFactory` 根据 category 从 Map 中获取对应 Validator； - 新增板块仅需添加 Validator 实现类并注册到 Map，无需修改 `DemandService` 和已有 Validator； 2. 订单状态：引入**策略模式** - 定义 `IOrderStateStrategy` 接口，为 6 种状态各实现一个策略类； - `OrderStateContext` 持有当前策略； - 新增状态仅需添加策略实现类并注册映射，无需修改 `OrderService`。

三、里氏替换原则（Liskov Substitution Principle, LSP）

检查问题

子类是否可无差别替换父类使用?

检查项	内容
AI 是否违反	是（1 处）
违反说明	`UrgentDemand extends Demand` 重写 `getExpiryTime()`，返回加急截止时间而非普通截止时间；`DemandService.listDemands()` 排序时依赖该方法，导致加急需求排序行为与普通需求不一致（排至列表末尾），破坏父类方法契约（返回「过期时间」用于排序）。
修正方案	用**组合替代继承**： 1. 移除 `UrgentDemand` 子类； 2. 在 `Demand` 实体中新增 `isUrgent`（Boolean）、`urgentDeadline`（LocalDateTime）字段； 3. `DemandFactory` 根据 `isUrgent` 决定是否设置加急截止时间； 4. Service 层通过自定义 Comparator 统一处理排序逻辑，不再依赖继承。

四、接口隔离原则（Interface Segregation Principle, ISP）

检查问题

是否存在接口过于「臃肿」，包含实现类无需的方法？

检查项	内容
AI 是否违反	是（1 处）
违反说明	定义 `ICampusService` 大接口，整合用户 / 需求 / 订单 / 评价管理方法；`UserServiceImpl` 被迫实现 `createDemand()`、`submitReview()` 等无关方法，只能空实现或抛 `UnsupportedOperationException`。
修正方案	拆分细粒度接口：`UserService`、`DemandService`、`OrderService`、`ReviewService`、`ArbitrationService`、`NotificationService` 等，每个接口仅包含自身领域的业务方法；实现类仅实现所需接口，无需处理无关方法。

五、依赖倒转原则（Dependency Inversion Principle, DIP）

检查问题

高层模块（Service）是否直接依赖低层模块的具体实现？

检查项	内容
AI 是否违反	是（1 处）
违反说明	1. `OrderService` 通过 `new NotificationServiceImpl()` 直接创建通知服务实例，未使用依赖注入； 2. 跨模块场景下，`OrderService` 直接注入 `DemandRepository`（其他模块数据访问层），绕过 `DemandService`，违反「跨模块必须走 Service 接口，禁止直接调用 Repository」的开发规范。
修正方案	1. 依赖注入：`OrderService` 对 `NotificationService` 的依赖改为 Spring 接口注入（`@Autowired private INotificationService notificationService`），由容器运行时注入实现； 2. 跨模块访问：改为调用 `DemandService` 公开方法，禁止直接注入其他模块的 Repository。

六、违规统计汇总

统计维度	数量
AI 原始设计违反 SOLID 总处数	7 处

单一职责（SRP）	2 处
开闭原则（OCP）	2 处
里氏替换（LSP）	1 处
接口隔离（ISP）	1 处
依赖倒转（DIP）	1 处
已修正处数	7 处

七、修正手段汇总

违反原则	核心修正手段
单一职责（SRP）	拆分 Service：抽出 CreditService、NotificationService、TagService
开闭原则（OCP）	1. 需求创建：工厂模式（DemandFactory + DemandValidator 接口 + 4 个校验实现类）； 2. 订单状态：策略模式（IOrderStateStrategy 接口 + 6 个状态实现类 + OrderStateContext）
里氏替换（LSP）	组合替代继承：Demand 实体内嵌 isUrgent/urgentDeadline 字段，移除 UrgentDemand 子类

接口隔离（ISP）	接口拆分：各 Service 独立定义接口，取消共用大接口
依赖倒转（DIP）	1. 依赖注入：改用 Spring @Autowired 注入接口； 2. 跨模块调用：通过 Service 公开方法，禁止直接注入 Repository